

Pipes: Passes the output of one command as input to another.

```
$ ls | grep "data"
data.csv
```

These tools empower users to chain commands, filter outputs, and automate complex tasks with elegance and efficiency.

Essential Commands and Utilities

The command line boasts a plethora of commands designed for various tasks. Here, we explore some of the most indispensable utilities.

File Operations

Managing files is a routine yet crucial aspect of system administration.

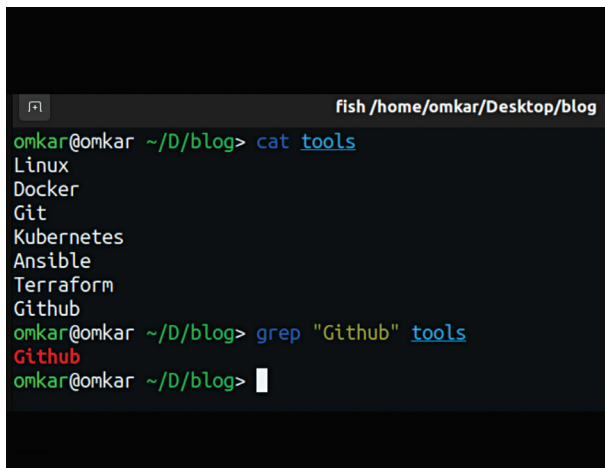
- **cp (Copy):** Duplicates files or directories.
\$ cp source.txt destination.txt
- **mv (Move/Rename):** Moves or renames files or directories.
\$ mv oldname.txt newname.txt
- **rm (Remove):** Deletes files or directories.
\$ rm unwanted.txt
- **touch:** Creates an empty file or updates the timestamp of an existing file.
\$ touch newfile.txt

These commands form the bedrock of file management, facilitating the organization and maintenance of data.

Text Manipulation

Processing and analyzing text data is a common requirement.

- **grep:** Searches for patterns within files.
\$ grep "error" logfile.txt
- **sed:** Stream editor for filtering and transforming text.
\$ sed 's/old/new/g' file.txt
- **awk:** Pattern scanning and processing language.
\$ awk '{print \$1}' data.csv
- **cut:** Removes sections from each line of files.
\$ cut -d',' -f1 data.csv



```
fish /home/omkar/Desktop/blog
omkar@omkar ~/D/blog> cat tools
Linux
Docker
Git
Kubernetes
Ansible
Terraform
Github
omkar@omkar ~/D/blog> grep "Github" tools
Github
omkar@omkar ~/D/blog>
```

These utilities enable efficient parsing, searching, and modification of text data, streamlining command-line workflows.

Shell Scripting Basics

Scripting allows users to automate repetitive tasks, creating efficient and reproducible workflows.

Writing and Executing Scripts

A basic shell script consists of a series of commands saved in a file with a .sh extension. To create and execute a script:

1. Open a text editor and write the script.
#!/bin/bash
echo "Hello, World!"
2. Save the file as `script.sh` and make it executable:
\$ chmod +x script.sh
3. Run the script:
\$./script.sh
Hello, World!

Variables, Conditionals, and Loops

Shell scripts use variables, conditionals, and loops for dynamic execution.

Variables:

```
name="Alice"
echo "Hello, $name"
```

Conditionals:

```
if [ -f file.txt ]; then
    echo "File exists."
else
    echo "File not found."
fi
```

Loops:

```
for i in {1..5}; do
    echo "Iteration $i"
done
```

Automating Tasks with Cron and systemd Timers

Cron Jobs: Used for scheduling recurring tasks.

```
$ crontab -e
0 5 * * * /path/to/script.sh
```

systemd Timers: Alternative to cron for scheduling.

```
$ systemctl enable myscript.timer
```

By leveraging shell scripting, users can automate complex workflows, reducing manual intervention and increasing efficiency. 